Statement of Qualifications

Uriah Raffer | JC-522483 | Information Technology Associate

1. Describe your experience installing, configuring, and maintaining Windows and/or Linux servers in an enterprise environment.

I install, configure, and maintain Windows and Linux servers across physical, virtual, and cloud infrastructure. In my security internship, which runs on Microsoft Azure and Tenable, I work hands-on with Windows Server, Windows 10 and 11, and Linux hosts in an enterprise environment, where I set up and configured a Windows Server myself and monitor the health of the systems I support.

I built those skills from the ground up in a homelab I run as a live network rather than a sandbox. I stood up a Windows Server 2022 domain, installing the operating system, promoting two domain controllers, DC01 and DC02, joining Windows 11 clients, and configuring Active Directory (AD) with separate Finance and Sales organizational units and a sixteen-character password policy. I set up Domain Name System (DNS) and Dynamic Host Configuration Protocol (DHCP) for the domain, with forward and reverse lookup zones, so it runs end to end the way a production domain does.

On the Linux side, I run several servers on a Proxmox hypervisor that keep my home network going, including two Pi-hole instances for DNS filtering and a pfSense firewall I use as a Tailscale exit node and subnet router. I install each operating system from a downloaded image, size its storage and memory, and configure its services. I built all of it myself to learn server administration by running it, not by reading about it.

2. Describe your experience implementing or supporting backup, recovery, or high-availability solutions for servers.

My experience implementing high-availability solutions comes from a cloud project I built and a Windows domain I run at home.

My High-Availability AWS Infrastructure project runs across two availability zones (AZ) on Amazon Web Services (AWS), built with Terraform and designed to stay up when a component fails. It runs a Multi-AZ PostgreSQL database that fails over to a standby in the second zone. To cut cost, I replaced the managed network address translation (NAT) gateway with cheaper self-managed instances. The NAT gateway recovers from a failure automatically. A self-managed instance doesn't. When one is replaced, it comes up with a new network interface while the private route still points at the old one, so outbound traffic breaks. I closed that gap with a self-healing route. An EventBridge rule detects the replacement and triggers a Lambda function that repairs the route to point at the new instance, so outbound traffic recovers on its own, the same resilience the NAT gateway provides at lower cost. I monitor the environment with CloudWatch and log every change to CloudTrail.

I did not assume it worked. I designed and ran a failure test that terminates the four instances, a NAT and a private one in each of the two zones, in every combination. That is fifteen scenarios, from a single failure up to all four at once, and the test times each recovery. Across three full runs, forty-five scenarios in all, every combination recovered without intervention, averaging about two and a half minutes. The route-repair function fired in under a second on each replacement, and one run logged ninety-two events

with fifteen route repairs and zero false positives. I tested the database the same way. Using a script that forced its Multi-AZ failover while a probe wrote to it continuously, I timed how long writes were refused. Across more than ten forced failovers, write access came back in about ten to thirteen seconds, and in one run only six of nearly four hundred writes failed during the cutover.

I also built and tested redundancy in my homelab domain. I configured DHCP failover across two domain controllers, then proved it by powering one off and watching address assignment carry over to the second without interruption. That second controller also holds a replicated copy of the primary's DNS zones, so identity, addressing, and name resolution all stay available if one goes down.

Both came from the same habit. Build for the failure you hope never comes, then test it to be sure it holds.

3. What are your procedures for managing server vulnerabilities, including patch management, vulnerability scanning, and incident response?

My procedure is a loop. Scan, prioritize, remediate, then verify, and keep records the whole way.

For vulnerability scanning, I run credentialed scans with Tenable so the scanner sees the system the way an authenticated user would, which surfaces the real findings instead of guesses. On one Windows Server I started at about thirty findings, worked them from the most severe down, and brought the system to three over several rounds, clearing every critical-severity finding by the second scan. I script the repeatable fixes in PowerShell so each one lands the same way every time.

Verification is part of the procedure, not an afterthought. After each fix I re-scan to confirm it took hold. That habit caught two things. Microsoft Edge kept flagging after I patched it, because the update only applied once the program was closed and reopened, so I learned to confirm a fix instead of trusting the installer. A high-severity SQLite finding kept appearing and disappearing across scans, so before touching it I searched the host with PowerShell for SQLite binaries and libraries, found none, and documented it as a false positive.

For patch management, I treat patching as a standing routine rather than a one-time pass. On my Linux servers, unattended upgrades run automatically every day. I also wrote a script I run by hand for a full update and cleanup pass, sometimes just to watch the text scroll by, even though the automated job has already handled it. My Windows systems update automatically as well.

For incident response, I investigate before I react. Reviewing sign-on activity in Microsoft Sentinel, I ran a Kusto Query Language (KQL) query that surfaced roughly one hundred failed sign-ins against an account, confirmed the pattern was a brute-force attempt, and identified the source Internet Protocol (IP) address the attempts came from so it could be blocked.

4. Outline your experience in administering and securing servers within an enterprise-level IT environment.

Securing a server, to me, means auditing it against a known-good baseline, closing the gaps, and proving they are closed.

I harden Windows systems to Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) baselines. I run the audit in Tenable, use PowerShell to remediate the settings that fail, and re-scan each one to confirm it passes. I applied the same approach to a Windows Server, enabling Server Message Block (SMB) signing, requiring Network Level Authentication (NLA) for Remote Desktop, and restoring weak authentication settings to secure values so credentials aren't exposed on the wire.

On the administration side, I run the directory services an enterprise depends on. In my Windows Server domain I manage user and group accounts and their membership, enforce a strong password policy, and keep the directory and its DNS records correct so name resolution and logins stay reliable for every client.

I carry that discipline to servers I run in the cloud, matching the access model enterprises treat as best practice. In an AWS environment I built with Terraform, I removed Secure Shell (SSH) access entirely. There is no open port 22, no key pairs to manage or leak, no password to brute force, and no bastion host to exploit. The only way in is AWS Systems Manager Session Manager (SSM). Identity and Access Management (IAM) decides who can reach each instance, connections use short-lived credentials, and every session is logged for audit. I enforced the hardened version of the instance metadata service to block a known attack that steals an instance's credentials, scoped automation to least-privilege roles instead of a broad administrative identity, and kept secrets encrypted at rest with Ansible Vault. It is a personal lab, but these are the access and identity controls an enterprise is expected to enforce.